PalmSource Developer Conference 2004
FEBRUARY 10-12, 2004 SAN JOSE, CA



Presented By:
George Hoffman
Director, Applications and Services

palm POWERED

Palm OS® Cobalt Details

Multimedia, Graphics, and the Window Manager

The contents of and the technologies described in this presentation are provided to you and described "AS IS," without any warranty or representation of any kind. PalmSource hereby disclaims each and every other representation or warranty with respect thereto, including the implied warranties of merchantability, fitness for a particular purpose, accuracy and non-infringement. No representation or warranties are made that any product based in whole or in part on any specifications or information referenced herein will be free from defects or safe for use for its intended purposes.

PalmSource further reserves the right to modify in any respect any technology described or content provided herein without prior written notice.

Palm OS is a registered trademark and Palm Powered is a trademark of Palm Trademark Holding Company, LLC. Other brands may be trademarks of their respective owners. All other trademarks and tradenames used herein are the property of their respective owners.

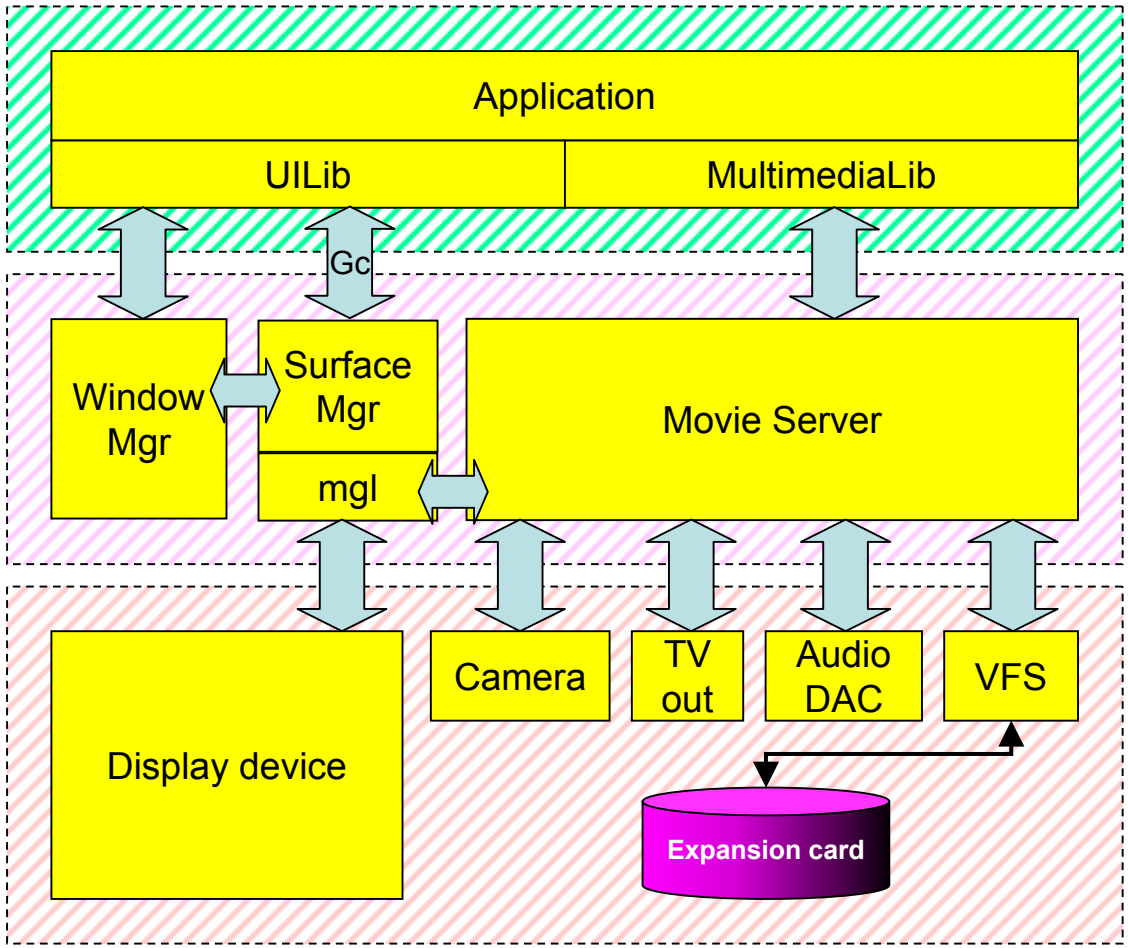
Copyright (c) 2004 PalmSource, Inc.

Architectural Overview

Application process

System process

I/O Subsystem

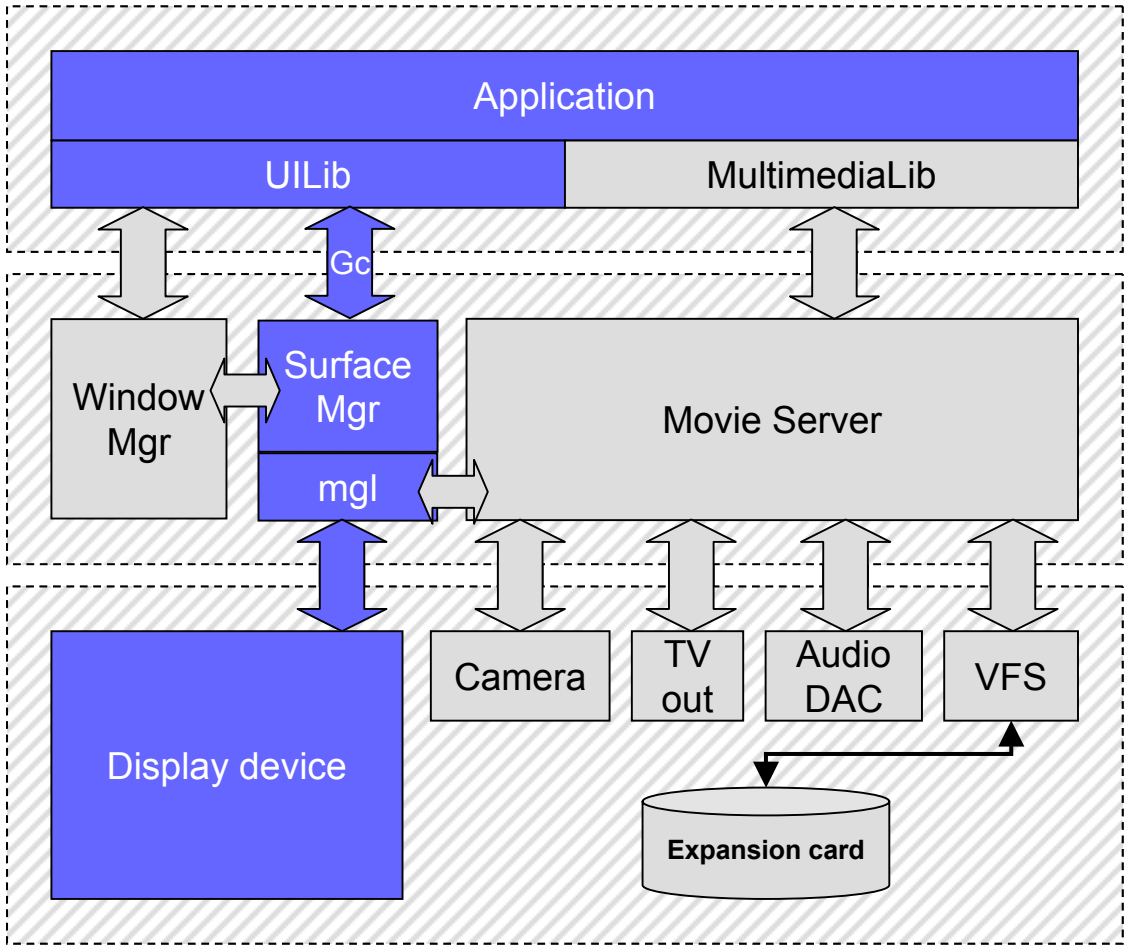


Graphics Services

Application process

System process

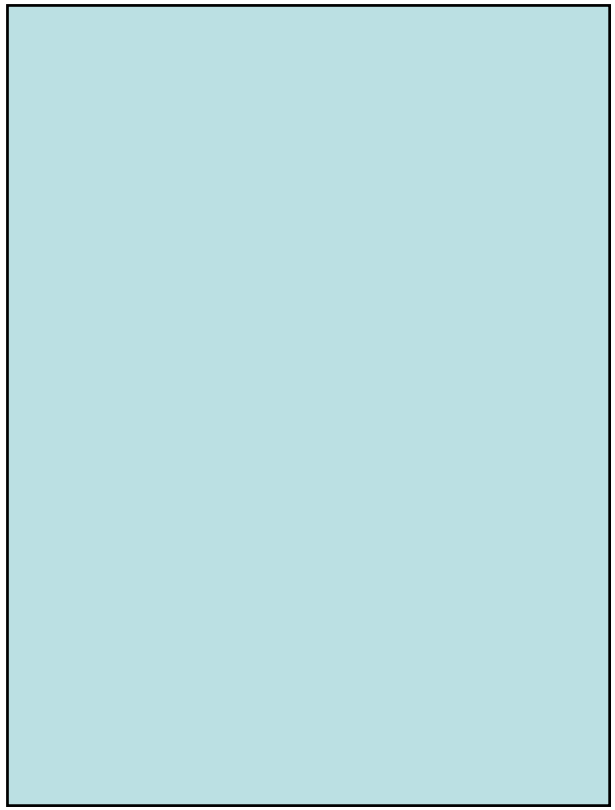
I/O Subsystem



Graphics Services

Gc Rendering APIs

- **Gc is for "Graphics Context"**
  - Explicit drawing window and state
  - API is independent from the `Win*` functions
  - Establishes a standard system 2D drawing model
- **Extensive feature set**
  - Dynamic presentation of rich content
  - Web browsers / e-book readers
  - Drawing and sketch applications
- **High performance**
  - Dynamic code generation
  - Designed for acceleration by modern 3D hardware





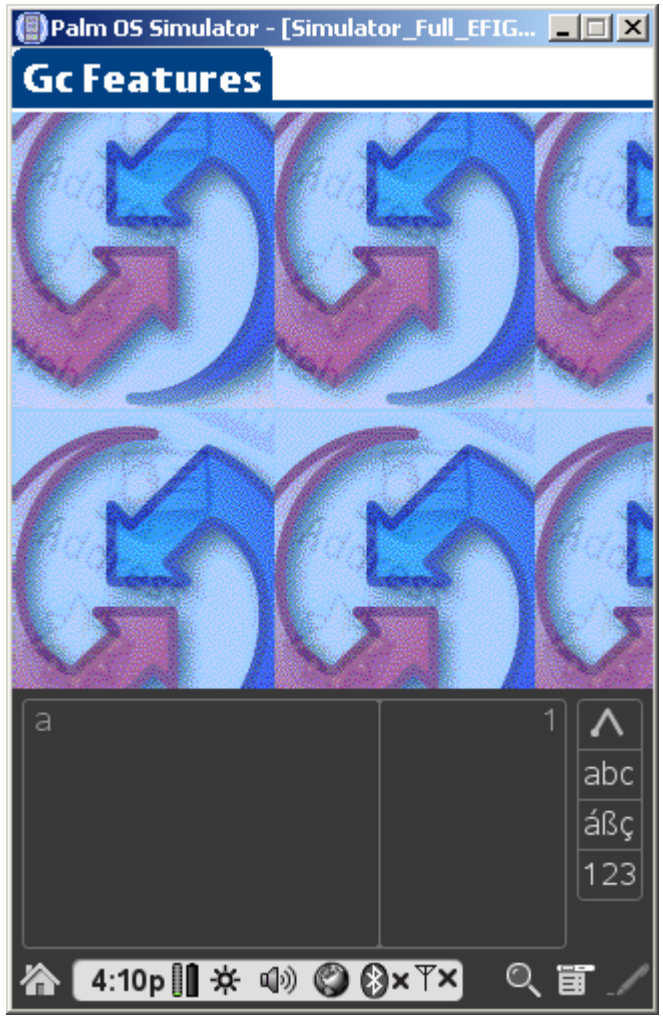
Graphics Services
The Graphics Context

- **Basic rendering state**
  - Solid color or gradient
  - 2D transform and coordinate system
  - Font
  - Line capping and joining
  - Winding rule
  - Anti-aliasing mode
- **Current shape**
  - Path (lines, arcs, bezier curves)
  - Clipping region
- **`GcPushState` and `GcPopState` push and pop entire context (state and shape)**

Graphics Services

Drawing Model

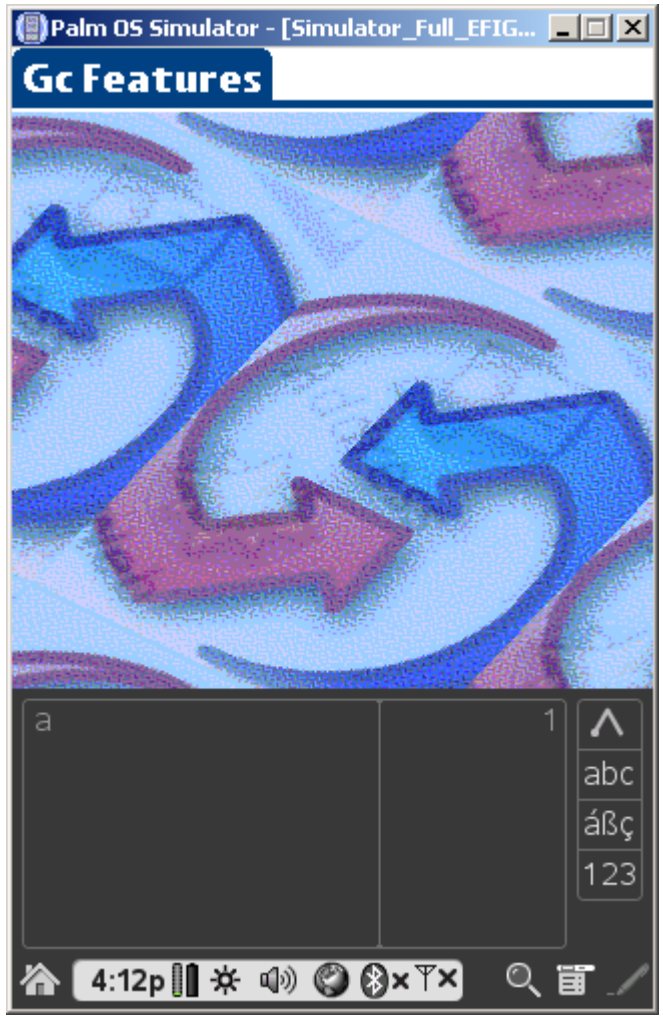
- **Path-based rasterization**



Graphics Services

Drawing Model

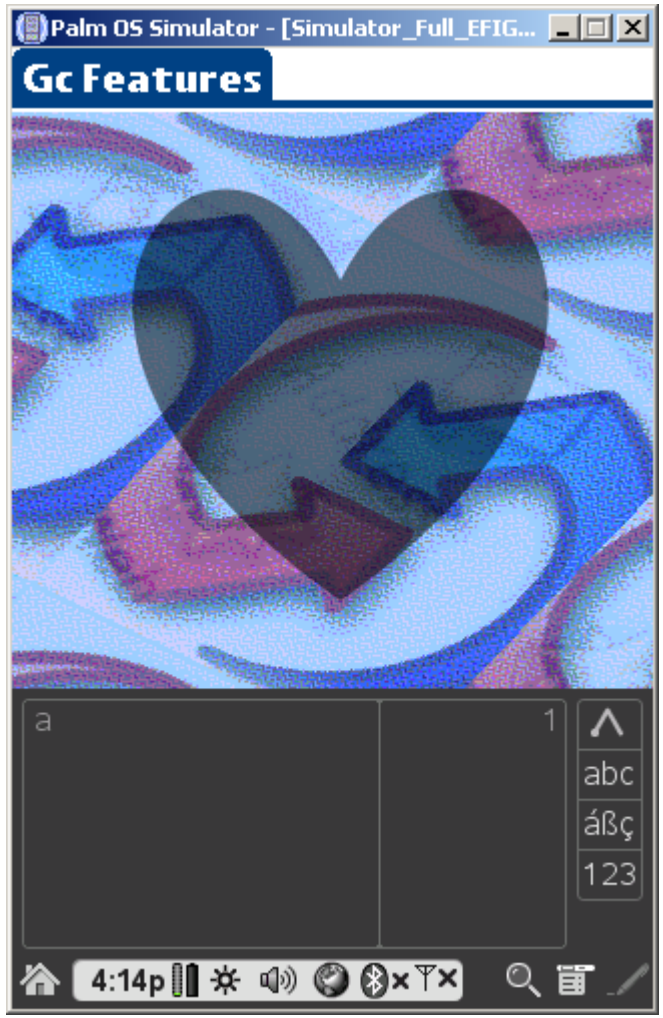
- **Path-based rasterization**
  - General 2D transformations



Graphics Services

Drawing Model

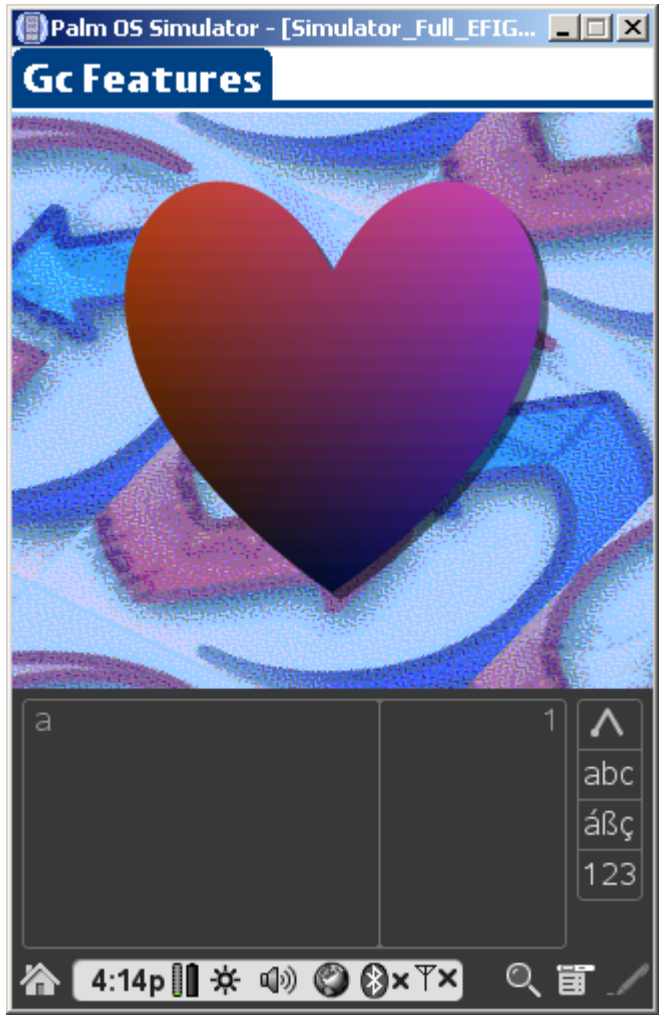
- **Path-based rasterization**
  - General 2D transformations
  - Arbitrary path filling
  - Alpha blending



Graphics Services

Drawing Model

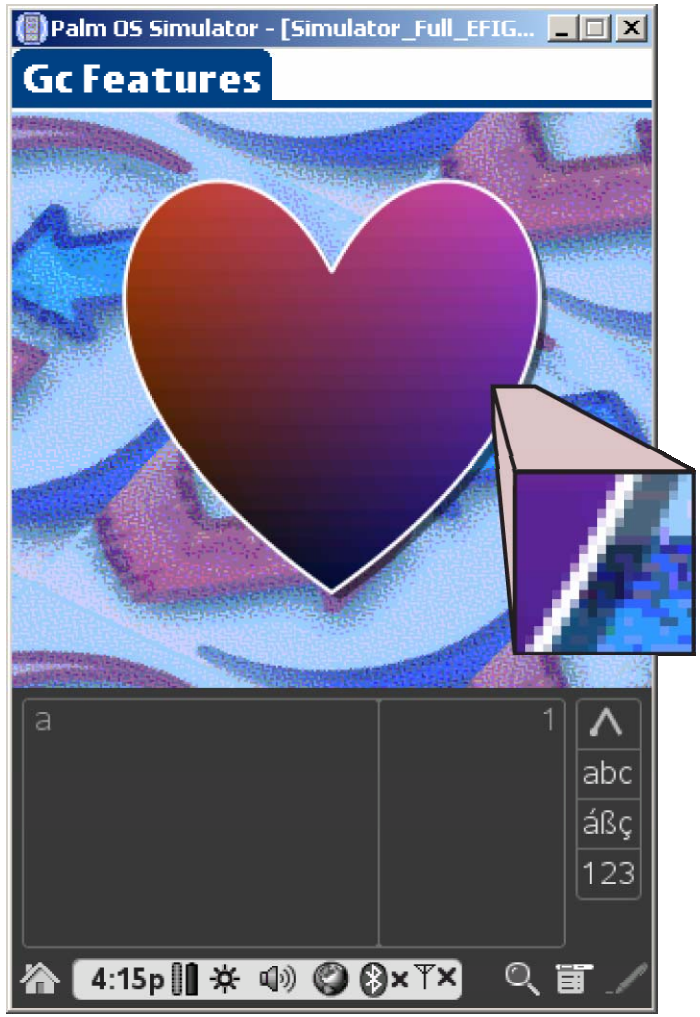
- **Path-based rasterization**
  - General 2D transformations
  - Arbitrary path filling
  - Alpha blending
  - Linear color gradients



Graphics Services

Drawing Model

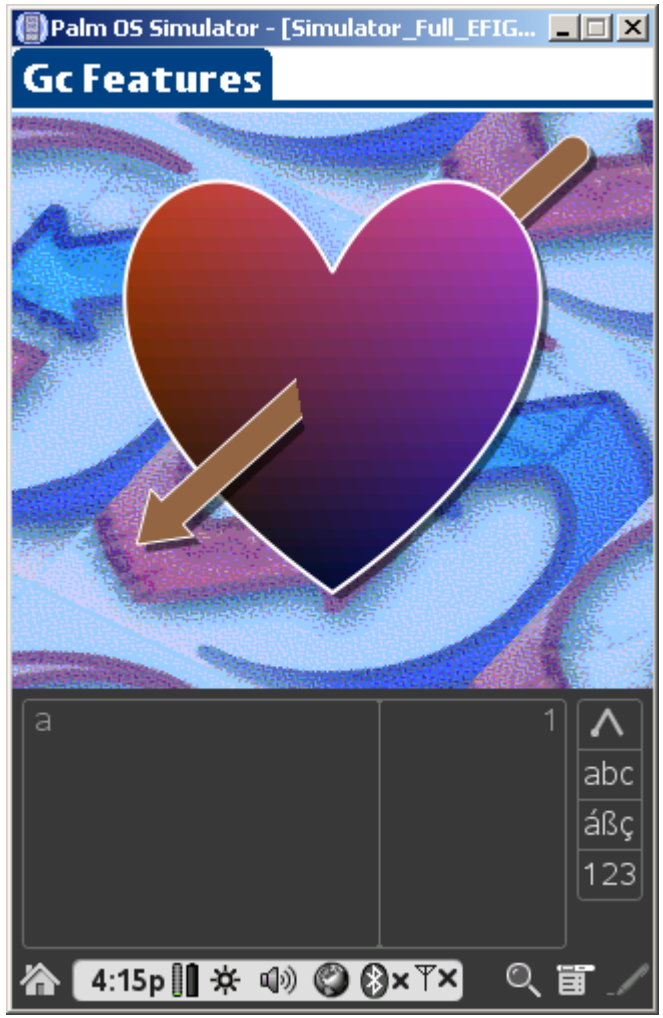
- **Path-based rasterization**
  - General 2D transformations
  - Arbitrary path filling
  - Alpha blending
  - Linear color gradients
  - Anti-aliasing



Graphics Services

Drawing Model

- **Path-based rasterization**
  - General 2D transformations
  - Arbitrary path filling
  - Alpha blending
  - Linear color gradients
  - Anti-aliasing
  - Path-based clipping



Graphics Services

Drawing Model

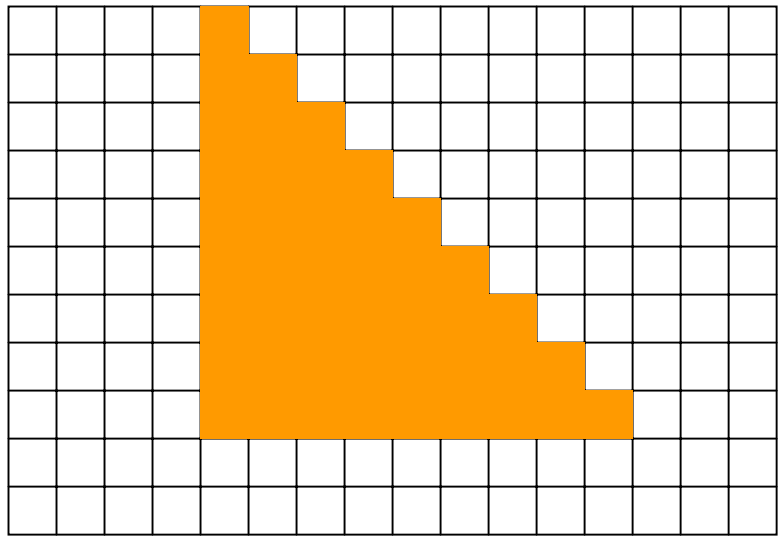
- **Path-based rasterization**
  - General 2D transformations
  - Arbitrary path filling
  - Alpha blending
  - Linear color gradients
  - Anti-aliasing
  - Path-based clipping
- **Scalable fonts**
  - TrueType font support
  - Easy to add new fonts



Graphics Services

Example: drawing an outlined shape

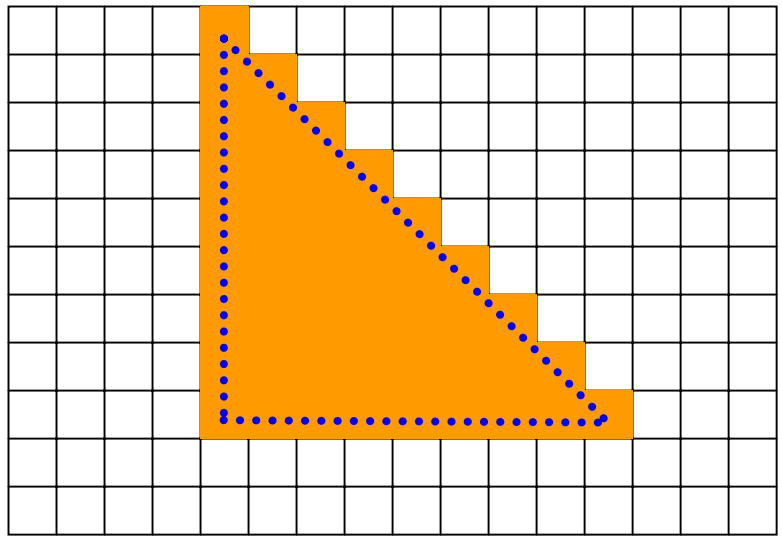
```
GcGetCurrentContext
GcSetColor( orange )
GcMoveTo, GcLineTo, GcLineTo, GcClosePath
GcPushState
GcPaint
GcPopState
GcSetColor ( green )
GcStroke
GcPaint
GcReleaseContext
```



Graphics Services

Example: drawing an outlined shape

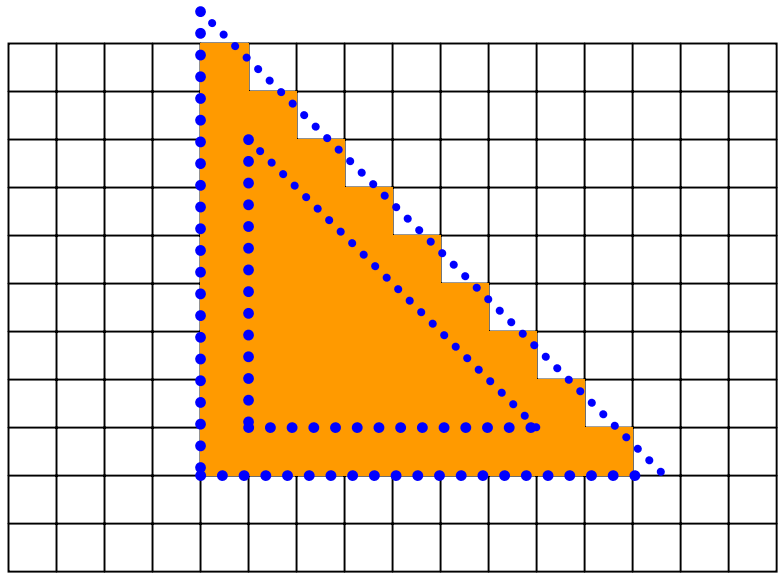
```
GcGetCurrentContext
GcSetColor( orange )
GcMoveTo, GcLineTo, GcLineTo, GcClosePath
GcPushState
GcPaint
GcPopState
GcSetColor ( green )
GcStroke
GcPaint
GcReleaseContext
```



Graphics Services

Example: drawing an outlined shape

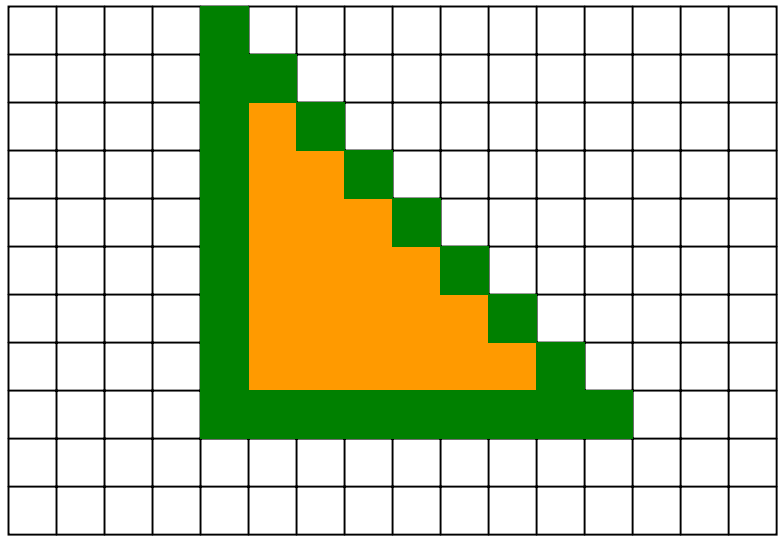
```
GcGetCurrentContext
GcSetColor( orange )
GcMoveTo, GcLineTo, GcLineTo, GcClosePath
GcPushState
GcPaint
GcPopState
GcSetColor ( green )
GcStroke
GcPaint
GcReleaseContext
```



Graphics Services

Example: drawing an outlined shape

```
GcGetCurrentContext
GcSetColor( orange )
GcMoveTo, GcLineTo, GcLineTo, GcClosePath
GcPushState
GcPaint
GcPopState
GcSetColor ( green )
GcStroke
GcPaint
GcReleaseContext
```





Graphics Services
Fonts

- **System-wide support for TrueType fonts**
  - Standard set of TrueType fonts available on system
  - More can be added through resource tool
- **Font substitution support**
  - A fall-back font is used for characters not present in the selected font
  - Useful for mixing Latin and non-Latin languages
- **New `GcFontHandle` for use with Gc**
  - Can represent scalable or bitmap fonts
  - Extensive font information/metrics available
  - Can't use scalable fonts in `Win*` APIs
- **Font picker app still uses bitmap fonts**



Graphics Services
Compatibility

- **Almost all `Win*` drawing APIs still work**
  - Implemented in terms of the new rendering model
  - Less efficient than using Gc API directly
  - A few “edge” features are lost
    - New model is true color
    - **`winInvert`** and **`winSwap`** limited to filled (round) rects
    - **`WinPaintBitmap`** only supports **`winOverlay`** mode
- **Using old APIs will modify Gc draw state**
  - Use **`GcPushState`**/**`GcPopState`** to protect Gc state
- **Do NOT change draw window while holding a Gc**



Graphics Services
References

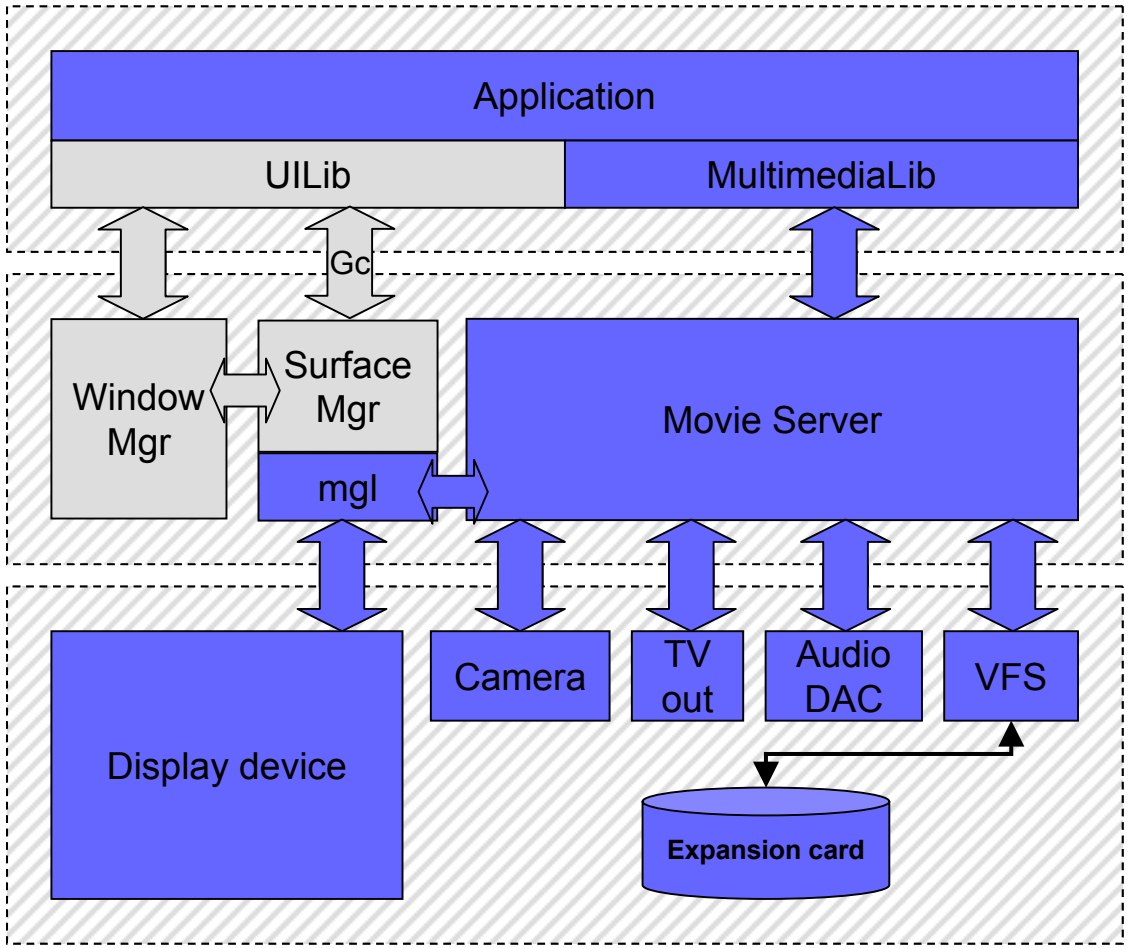
- **Documentation**
  - *Exploring Palm OS: User Interface*
- **Headers**
  - `GcRender.h`
  - `GcFont.h`

Multimedia Services

Application process

System process

I/O Subsystem



Multimedia Services

MultimediaLib APIs

- **Application-level API for controlling media tasks from a Palm OS Cobalt application**
- **Allows applications to create and communicate with Movie Server objects:**
  - Sessions
  - Sources and Destinations
  - Tracks
- **Allows applications to make use of extensions:**
  - Session Classes
  - Device Properties
  - Codecs





Multimedia Services
Sessions

- **A session represents a type of media task**
- **Applications create sessions, and choose…**
  - the *session class* (a built-in or licensee-provided component type)
  - whether to host the session locally (in the application process) or remotely in a background process
- **Applications configure sessions by adding…**
  - Sources and Destinations
  - Tracks
- **Standard built-in session classes are provided for playback and capture (recording) of audio and video**



Multimedia Services

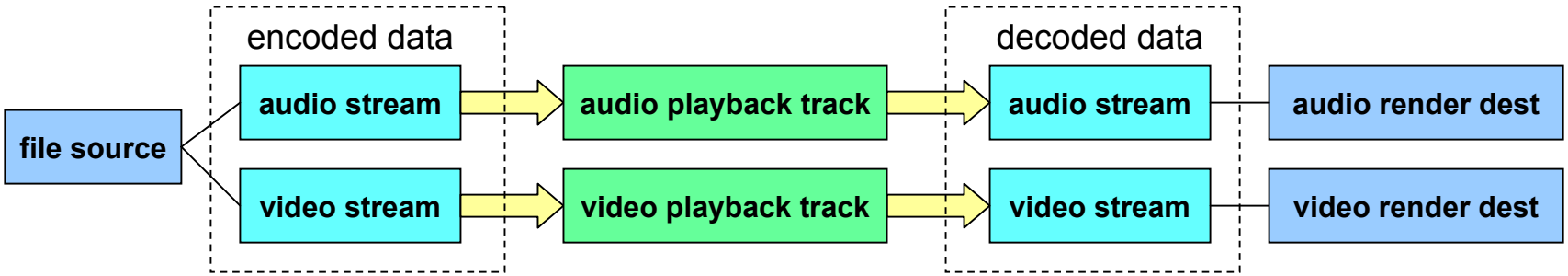
Sources and Destinations

- **Every session allows the specification of one or more media sources and one or more media destinations**
- **Sources and destinations are primarily specified by URL**
  - Example: *file:///Card1/Audio/MySong.mp3*
  - Example: *palmdev:///Media/Default/VideoOut*
  - Constants are defined for default media devices
- **They represent encoded data files or devices that produce or accept media data**
  - A stored bytestream like a file or a network socket
  - A real device like a microphone, camera or TV-out
  - A virtual device representing a playback window
- **Each source or destination is designed to contain one or more *streams***
  - A movie typically contains an audio stream and a video stream
  - Applications may choose to playback or capture individual streams, or may configure the session to automatically use all applicable streams

Multimedia Services

Tracks

- **A track represents a data-processing route from a source stream to a destination stream**
- **Every session needs at least one track in order to do something**
- **Applications can specify which tracks they care about, or use all applicable tracks**





Multimedia Services
Properties

- **Properties are attached to many objects**
  - Sessions
  - Sources and destinations
  - Streams
  - Tracks
- **Provide enumeration, getters, and setters for parameters that control session behavior**
  - Encoding formats and options
  - Playback location
  - Device-specific parameters (e.g. camera flash on/off)
- **Extensible with hardware-specific properties**



Multimedia Services

Example: start a playback session

```
MMSessionID session;
MMSourceID source;

/* Create a standard playback session */
MMSessionCreate(P_MM_SESSION_CLASS_DEFAULT_PLAYBACK, 0, &session);

/* Attach a source to it */
MMSessionAddSource(session, file:///CARD1/mySong.mp3, &source);

/* Hook up the "default" track(s) for the format (in this case, a
   single audio stream) */
MMSessionAddDefaultTracks(session, source, 0);

/* Begin playback */
MMSessionControl(session, P_MM_SESSION_CTL_RUN);
```



Multimedia Services
Filters

Demo



Multimedia Services
References

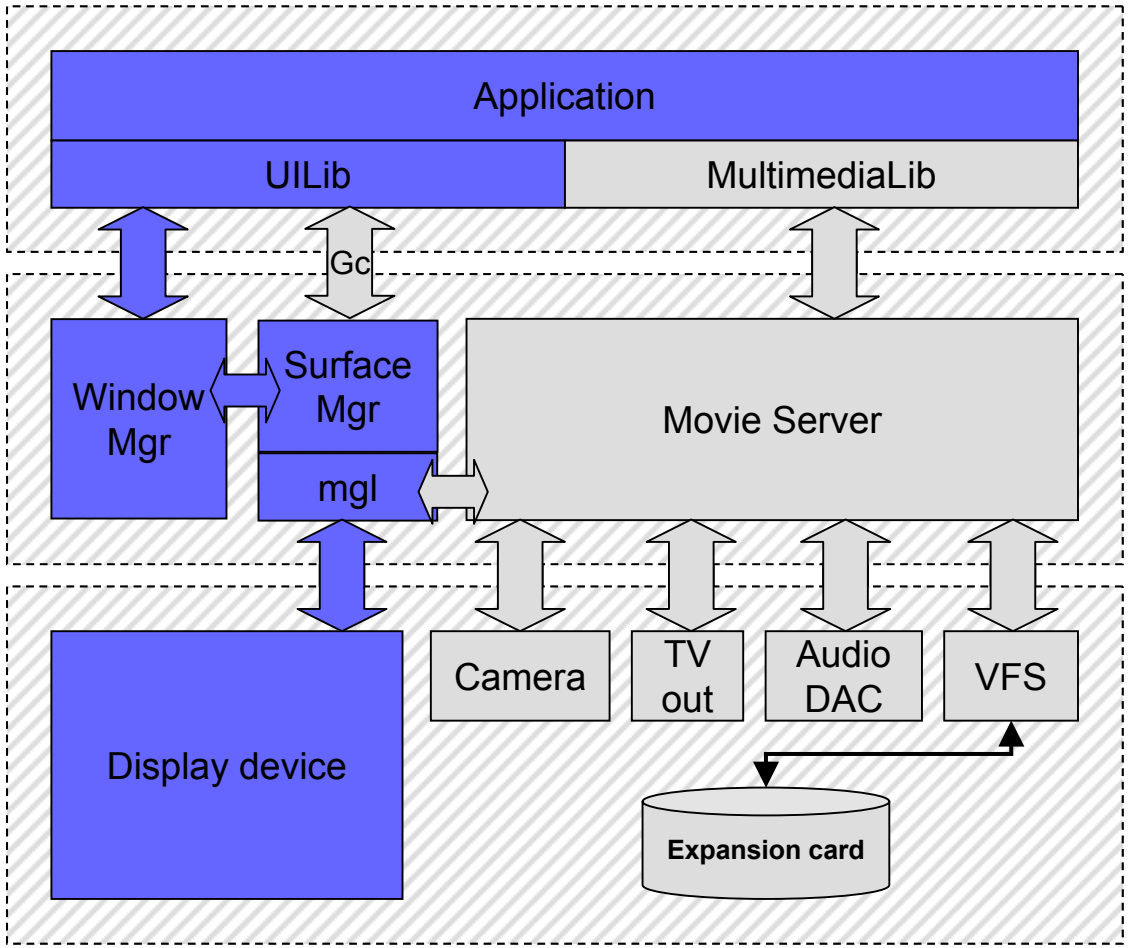
- **Documentation**
  - *Exploring Palm OS® : Multimedia*
- **Headers**
  - `Multimedia.h`
  - `MMDefs.h`
  - `MMCodecClass.h`
  - `MMFormat.h`
  - `MMProperty.h`
  - `MMSession.h`
  - `MMSessionClass.h`
  - `MMTrack.h`

Window Manager

Application process

System process

I/O Subsystem





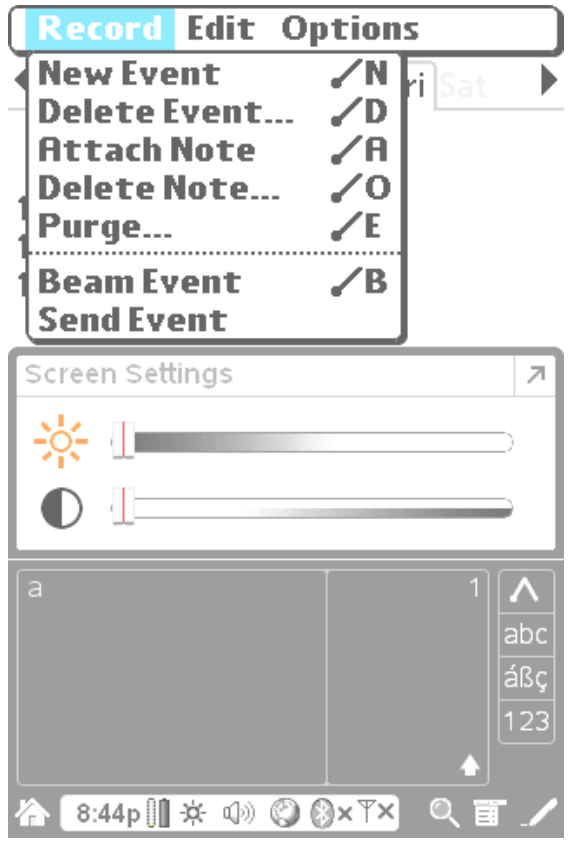
Window Manager
Overview

- **Palm OS® Cobalt is designed for multi-threaded UI**
- **The Window Manager has far more responsibility**
  - Only way to get access the screen (can't get framebuffer ptr)
  - Multiplexes the screen and input(s) among many UI threads
  - Enforces clipping to draw window
  - Decides on layout across a growing range of form factors
- **It strikes a deal with your application**
  - Your app gets more power and flexibility
  - The Window Manager gets ultimate control over the screen
  - Resulting environment limits the intelligence and complexity required of any one application

Window Manager

Layers

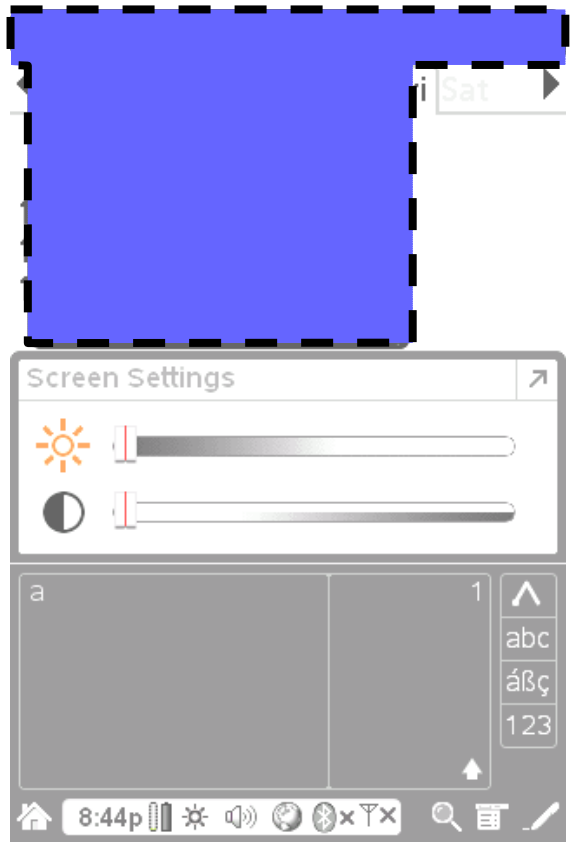
- **Multiplexing between UI threads**
  - Window manager enforces correct clipping regions
  - Layers enforce specific depth ordering
  - Windows in the same layer stack, most recent on top
- **Select layer with window flags**
- **Current layers (front to back) are…**
  - `winLayerMenu`
  - `winLayerSystem`
  - `winLayerSecurity`
  - `winLayerPriority`
  - `winLayerSlip`
  - `winLayerNormal`



Window Manager

Layers

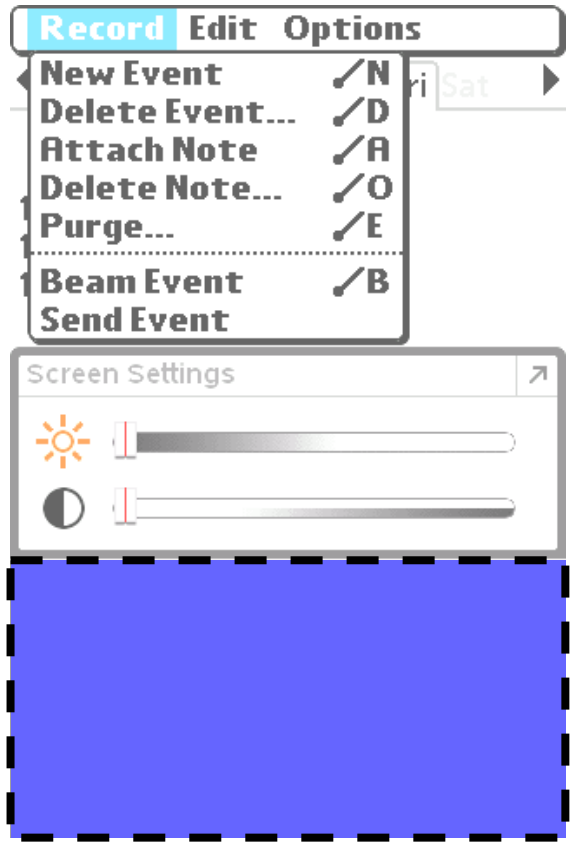
- **Multiplexing between UI threads**
  - Window manager enforces correct clipping regions
  - Layers enforce specific depth ordering
  - Windows in the same layer stack, most recent on top
- **Select layer with window flags**
- **Current layers (front to back) are…**
  - `winLayerMenu`
  - `winLayerSystem`
  - `winLayerSecurity`
  - `winLayerPriority`
  - `winLayerSlip`
  - `winLayerNormal`



Window Manager

Layers

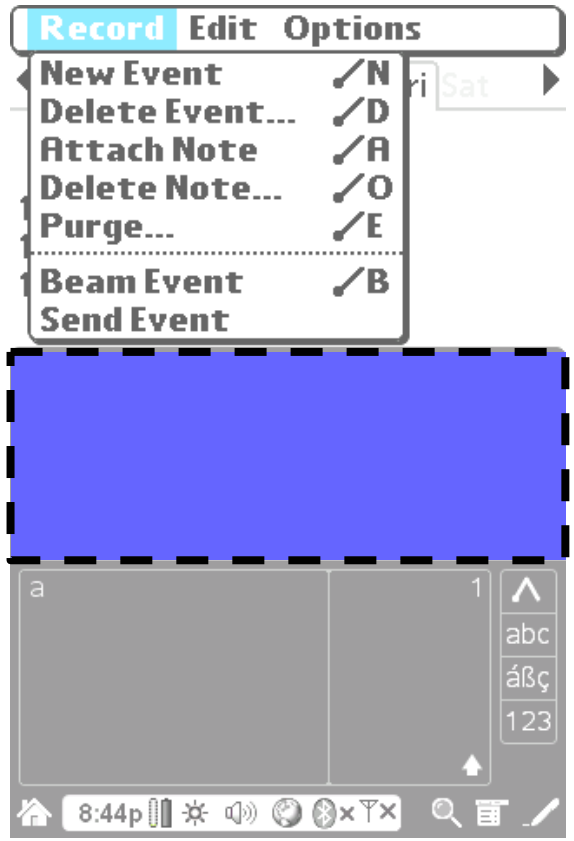
- **Multiplexing between UI threads**
  - Window manager enforces correct clipping regions
  - Layers enforce specific depth ordering
  - Windows in the same layer stack, most recent on top
- **Select layer with window flags**
- **Current layers (front to back) are…**
  - `winLayerMenu`
  - `winLayerSystem`
  - `winLayerSecurity`
  - `winLayerPriority`
  - `winLayerSlip`
  - `winLayerNormal`



Window Manager

Layers

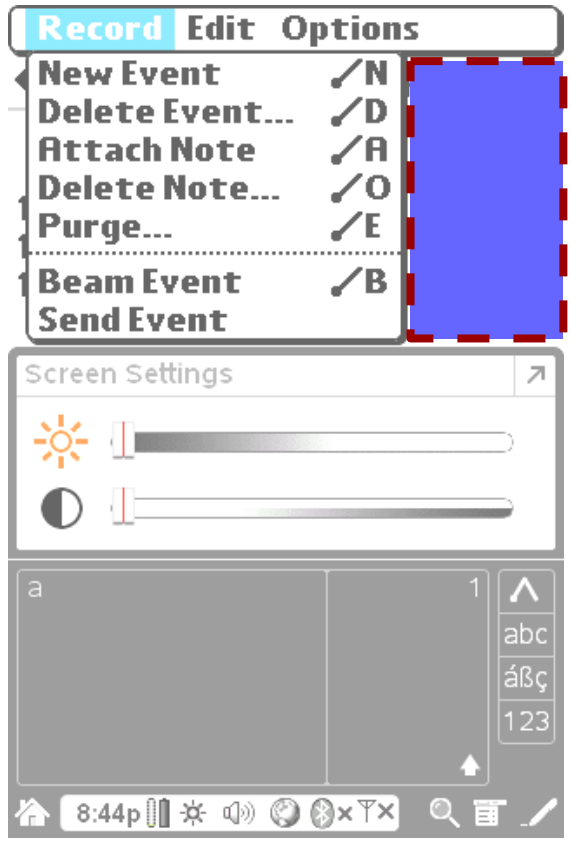
- **Multiplexing between UI threads**
  - Window manager enforces correct clipping regions
  - Layers enforce specific depth ordering
  - Windows in the same layer stack, most recent on top
- **Select layer with window flags**
- **Current layers (front to back) are…**
  - `winLayerMenu`
  - `winLayerSystem`
  - `winLayerSecurity`
  - `winLayerPriority`
  - `winLayerSlip`
  - `winLayerNormal`



Window Manager

Layers

- **Multiplexing between UI threads**
  - Window manager enforces correct clipping regions
  - Layers enforce specific depth ordering
  - Windows in the same layer stack, most recent on top
- **Select layer with window flags**
- **Current layers (front to back) are…**
  - `winLayerMenu`
  - `winLayerSystem`
  - `winLayerSecurity`
  - `winLayerPriority`
  - `winLayerSlip`
  - `winLayerNormal`





Window Manager
Modality

- **Window Manager enforces modality**
  - `winFlagModal`
  - `FrmLoadForm` sets this for you from form's modal flag
- **Modal windows eat all pen events**
  - Any events on windows behind them (in Z order) will go to the modal window
  - Non-modal windows above them still receive events
- **Modal windows can block keyboard focus from windows under them**
  - If the modal window has focus, a request for focus will be remembered but not granted until the modal window goes away
  - Modal windows can always take focus from windows under them



Window Manager

Layout

- **Your app and the Window Manager cooperate to place your windows**
  - Applications know what they can best support
  - The system knows the device and what else is going on
    - Various form factors and screen sizes
    - Other UI elements contending for the screen
    - User preferences
- **Windows declare their needs**
- **The system does its best to honor them**
- **Layout policies may vary from device to device**
- **Tools provided for automating simple form layout rules – `FormLayout.h`**



Window Manager

Constraints

- **Windows declare their layout constraints**
  - Constraints resource
  - `WinSetConstraints()`
  - `WinCreateWindowWithConstraints()`
- **Window Manager performs layout and sends `winResized` events to report the new size to any windows that are affected**
- **Resizing is asynchronous**
  - `WinSetBounds()` should be avoided
  - Just because you request a size doesn't mean you'll get it
  - Only re-layout your forms once you receive the event
  - Other windows (including yours!) may be affected, too
- **Your window can be resized at any time**
  - If you don't take it into account, you may be clipped or have unused space in your window



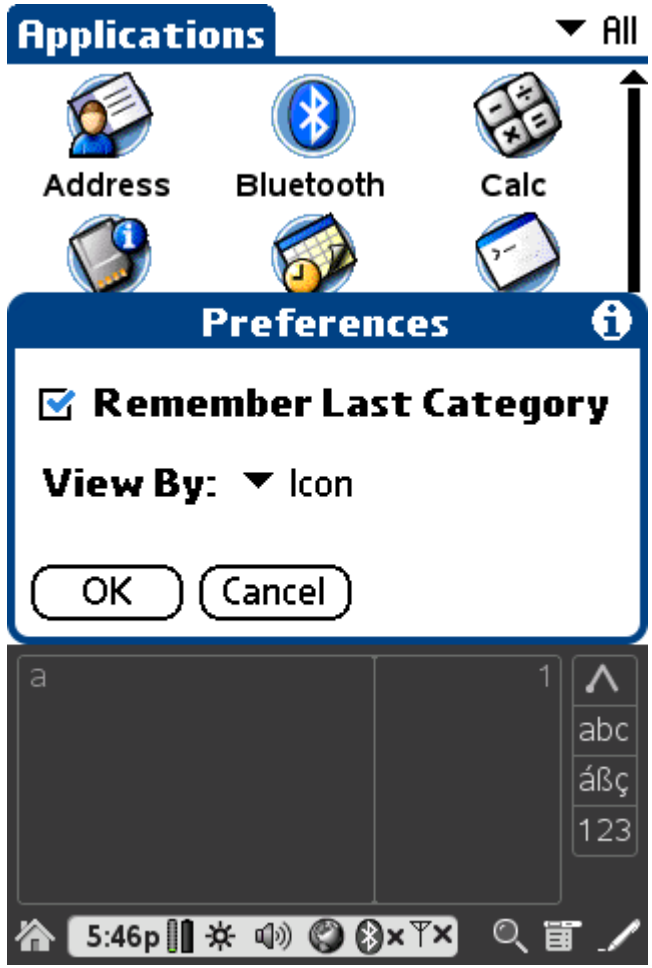
Window Manager
Updates

- **Constraints-based windows must draw when they receive update events**
  - Has been the case for all windows on Debug ROMs for a while now
  - `WinSaveBits/WinRestoreBits()` are forcefully deprecated
- **Drawing "outside of an update" is a bad practice**
  - You can't depend on what you drew last being unmodified
  - Forbidden under certain circumstances
  - Uses a back-buffer when it is supported
- **A window declares its behavior**
  - Legacy
  - Transitional
  - Update-based
- **Transitional windows are recommended**

Window Manager

Example: the input area closes

- **Main Form**
  - "I would like to be as tall as possible"
- **Dialog**
  - "I have a fixed size and can be placed anywhere"
- **Input area**
  - "I want to be fully open"



Window Manager

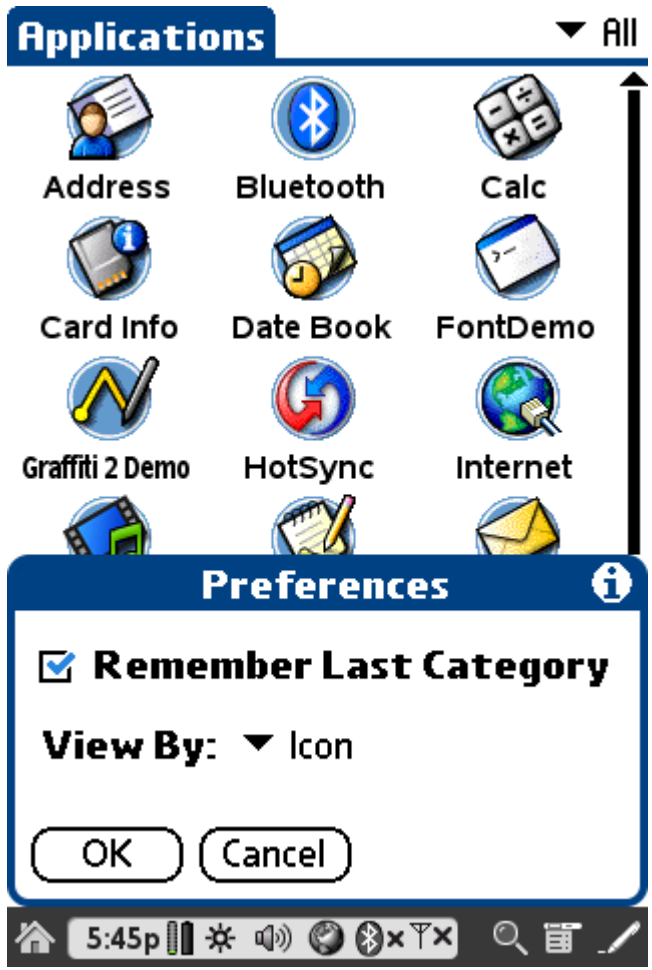
Example: the input area closes

- **Main Form**
  - "I would like to be as tall as possible"
- **Dialog**
  - "I have a fixed size and can be placed anywhere"
- **Input area**

  "I would like to be smaller"

  "I would like to be even smaller"

  "I would like to be closed"





Window Manager

Multithreaded UI

- **Main UI thread has full UI features available**
- **Other threads are more restricted**
  - They can only use update-based windows
  - No sub-launches or notifications
- **Guidelines for UI outside of main thread**
  - Use the priority layer (don't get buried by app)
  - Observe guidelines for update-based windows
- **Example: showing an alert in a worker thread**

```
DmOpenRef appdb;
SysGetModuleDatabase(SysGetRefNum(), NULL, &appdb);

WinStartThreadUI();
FrmAlertWithFlags(appdb, MY_ALERT_ID, winLayerPriority);
WinFinishThreadUI();
```



Window Manager
References

- **Documentation**
  - *Exploring Palm OS: User Interface*
  - *Exploring Palm OS: System Management*
- **Headers**
  - `Window.h`
  - `FormLayout.h`



Q&A



palm OS Cobalt

Contact: Doug MacMillan
Sr. Palm OS Platform Evangelist
Email: doug.macmillan@palmsource.com

Contest: Open Labs (Tues/Wed)
68K Compatibility & Protein Contests
Motorola sponsored Grand Prize!

MOTOROLA digital dna